

Document-level Relation Extraction & Knowledge Graph 시스템

연구 계획서 — Hybrid 구조 확장판

DocRED 기반 End-to-End 정보 추출 파이프라인 설계 및 구현

ATLOP + DREEAM + GAIN 구조 기반 3단계 Incremental Stacking 설계

2026.04.08.

1. 프로젝트 개요

본 프로젝트는 DocRED 데이터셋을 기반으로 문서 수준 관계 추출(Document-level Relation Extraction)을 수행하고, 추출된 관계를 그래프 구조로 확장하여 Knowledge Graph 및 Graph-based Reasoning 시스템으로 발전시키는 것을 목표로 한다. 기존 pipeline 구조에서 한 단계 나아가, SSAN의 구조 인식 인코딩과 GAIN의 이중 그래프 추론을 모델 내부에 통합한 Hybrid 구조를 핵심 설계 목표로 삼는다.

최종적으로 3단계 Incremental Stacking 방식으로 모델을 확장하여 각 모듈의 효과를 정량적으로 검증하고자 한다.

핵심 목표
단순 관계 분류 모델이 아닌 End-to-End 구조화 파이프라인 구현
문서 → 관계 추출 → 그래프 구성 → 추론 → 질의/응용 흐름 완성
NLP 모델 + 그래프 시스템을 결합한 통합 아키텍처 설계
SSAN 구조 인식 + GAIN 이중 그래프 + U-NET 을 모델 내부에 통합한 Hybrid 아키텍처

1.1 전체 파이프라인 계획

단계	내용
① Preprocessing	DocRED → Feature 변환
② Structure-aware Encoder	SSAN-lite: Transformer + Entity Structure Bias
③ Entity Representation	Mention → Entity 벡터 (logsumexp pooling)
④ Hybrid Relation Extraction	Direct Branch + Graph Branch → Fusion → Multi-label 분류
⑤ Postprocessing	Triple 정제 및 필터링
⑥ Knowledge Graph	Neo4j 그래프 구성
⑦ Graph Reasoning	GNN + Multi-hop 추론 ← 최종 출력

1.2 문제 정의

Document-level Relation Extraction 이란?

문서 수준 관계 추출(DocRE)은 하나의 문서 내에 등장하는 entity 쌍에 대해 어떤 관계가 존재하는지 예측하는 태스크이다. 기존 문장 수준 RE와의 핵심 차이는 다음과 같다.

항목	Sentence-level RE	Document-level RE (본 프로젝트)
입력 범위	단일 문장	전체 문서 (multi-sentence)
관계 단서	문장 내 로컬 정보	문서 전체 + inter-sentence 단서
출력 형식	단일 라벨 분류	Multi-label (복수 관계 허용)
추론 방식	직접 언급된 관계만	Multi-hop 간접 추론 포함

핵심 어려움

- Cross-sentence reasoning: 하나의 관계를 확인하는 단서가 여러 문장에 분산
- Multi-hop inference: $A \rightarrow B$, $B \rightarrow C$ 정보로부터 $A \rightarrow C$ 관계를 간접 추론해야 함
- Multi-label: 하나의 entity 쌍이 동시에 여러 관계를 가질 수 있음
- 클래스 불균형: 실제 관계(positive)에 비해 no_relation pair 가 압도적으로 많음
- Long-context: DocRED 문서는 평균 약 200 토큰 이상, 일부는 512 초과 → Longformer 필요

1.3 평가지표

본 프로젝트는 DocRE 커뮤니티 표준을 따르며, 아래 지표를 핵심 평가 기준으로 사용한다.

지표	수식	의미	목표 기준
Micro F1	$2PR / (P+R)$	모든 relation 예측을 동등 가중치로 평가. 표준 비교 지표.	$V1 \geq 55\%$ / $V2 \geq 60\%$
Ign F1	훈련-테스트 공유 triple 제외 F1	훈련/테스트 셋에 동시에 등장하는 triple을 제외한 엄밀한 평가.	V2 기준 Micro F1 대비 -3~5%p 이내
Evidence F1	근거 문장 예측 F1	DREEAM 확장 시 적용. 관계의 근거 문장을 얼마나 정확히 찾는지.	DREEAM 확장 단계에서만 측정
Intra-sent F1	문장 내 관계 F1	단일 문장 내에서 완결되는 관계. 모델의 기본 RE 능력 측정.	V3 Hybrid에서 Direct Branch 기여 확인
Inter-sent F1	문장 간 관계 F1	복수 문장 단서가 필요한 관계. Graph Branch 기여 측정에 핵심.	V3 Hybrid에서 Graph Branch 기여 확인

단계별 목표 성능

실험 단계	모델 구성	Micro F1 목표	추가 지표
Stage 01	BERT + mean pooling + pairwise classifier	$\geq 55\%$	Ign F1
Stage 02	Stage 1 + ATLOP + DREEAM	$\geq 60\%$	Ign F1
Stage 03	Stage 2 + U-Net	V2 대비 향상	-
Stage 04	Stage 2 + GNN (GAIN-lite)	V2 대비 향상	-

2. 기존 논문 성능 비교 (DocRED 벤치마크)

아래는 DocRED 테스트셋 기준으로 보고된 주요 선행 논문의 성능이다. 본 프로젝트의 각 실험 단계는 이 논문들과 직접 비교 가능한 구조로 설계한다.

모델 (논문)	인코더	Dev F1	Dev Ign F1	핵심 기여
GAIN (Zeng 2020)	BERT-base	57.28	55.08	Double Graph
SSAN (Xu 2021)	RoBERTa-large	65.92	63.78	Structure Attention
ATLOP (Zhou 2021)	RoBERTa-large	63.40	61.32	Adaptive Threshold
DREEAM (Ma 2023)	RoBERTa-large	67.53	65.31	Evidence-guided Attn
본 프로젝트 Baseline (목표)	BERTa -base	-	측정 예정	Pipeline 구축
본 프로젝트 ATLOP style + Dreeam	BERTa-base	≥ 60%	측정 예정	ATLOP 방식 (Dreeam 확장)
본 프로젝트 structure Hybrid	BERTa -base + GNN / U-net	≥ 60%	측정 예정	Hybrid 구조

※ Dev F1 / Ign F1 수치는 각 논문 원문 보고값 기준. 본 프로젝트는 BERTa-base를 사용하므로 large 모델 대비 수치가 다를 수 있음.

3. 최종 목표 및 4 단계 확장 로드맵

목표 요약
① Document-level Relation Extraction 수행 (multi-label 관계 예측)
② Multi-hop reasoning 기반 구조 설계
③ Knowledge Graph 확장 가능 데이터 생성
④ Baseline → Hybrid 구조로 확장하여 각 모듈 효과를 정량 검증

왜 Baseline → Hybrid 순서로 가야 하는가?

Hybrid 구조는 multi-hop 추론을 relation 예측에 직접 반영할 수 있어 잠재 성능이 높다. 그러나 비교 기준(baseline) 없이 바로 Hybrid만 실험하면 아래 문제가 발생한다.

- 성능 귀인 불가: SSAN-lite 덕분인지, GNN 덕분인지, fusion 방식 덕분인지 구분 불가
- 디버깅 난이도 급증: 전처리·encoder·graph edge·GNN·fusion 중 어디가 문제인지 추적 어려움
- 보고서 설득력 약화: Baseline → 확장 → 비교 흐름이 없으면 Hybrid의 효과를 정량 제시 불가

4. 4 단계 Incremental Stacking 로드맵

기존 보고서의 sequential pipeline (Encoder → Entity Repr → Relation Extraction)을 확장하여, SSAN의 구조 인식 인코딩과 GAIN의 이중 그래프 추론을 모델 내부에 통합한다.

4.1 전체 모델 흐름

Stage 1: Baseline RE

```
Encoder (BERT-base)
→ Entity Representation (mention pooling)
→ Pair-wise Classifier (bilinear + sigmoid)
→ relation_logits
```

Stage 2: Stage 1 + ATLOP + DREEAM

```
Encoder (BERT-base)
→ Entity Representation (LogSumExp pooling)    ← 변경
→ ATLOP Classifier (adaptive threshold)        ← 변경
→ relation_logits
→ evidence_logits                             ← 추가 (DREEAM)
```

Stage 3: Stage 2 + Structural Reasoning (GAIN-lite)

```
Encoder (BERT-base)
→ Entity Representation (LogSumExp pooling)
→ [전역 구조 추론 계층]                        ← 추가
    이기종 GNN: Entity-Sentence-Document 3층 그래프
→ Refined Entity Vectors
→ ATLOP Classifier (adaptive threshold)
→ relation_logits + evidence_logits
```

Stage 4: Stage 2 + Structural Reasoning (GAIN-lite)

```
Encoder (BERT-base)
→ Entity Representation (LogSumExp pooling)
→ [전역 구조 추론 계층]                        ← 추가
    U-Net: N×N 2D 쌍 행렬 + CNN
→ Refined Entity Vectors
→ ATLOP Classifier (adaptive threshold)
→ relation_logits + evidence_logits
```

4.2 Structured Graph 아이디어

4.2.1 구조 인식 인코더 (SSAN-lite)

SSAN 핵심 아이디어 (Xu et al. 2021)

SSAN(Structured Self-Attention Network)은 Transformer 인코더의 각 self-attention 블록 내부에 entity 구조 의존성(coreference / co-occurrence / no-relation)을 bias로 주입한다. 이를 통해 인코더가 처음부터 entity 간 구조를 인지한 상태에서 contextual representation을 생성한다.

mention pair 유형	SSAN 원본	SSAN-lite (본 프로젝트)
Coreference	Biaffine 또는 Decomp transform	Biaffine transform 단일 사용
Co-occurrence	같은 entity 내 mention	동일 처리
No-relation	entity 무관 token pair	bias = 0
구현 위치	모든 BERT self-attention layer	상위 N layer만 적용 (속도 최적화)

논문 근거: Xu et al. (2021) SSAN Dev F1 65.92 (RoBERTa-large) — GitHub:
<https://github.com/BenfengXu/SSAN>

4.2.2 이중 그래프 추론 (GAIN Double Graph)

GAIN 핵심 아이디어 (Zeng et al. 2020)

GAIN은 두 개의 그래프를 동시에 구성하여 local 문맥과 global 관계를 모두 포착한다. 기존 보고서의 Graph Reasoning Layer가 후처리에 가까웠던 것과 달리, Hybrid 구조에서는 이 그래프를 relation 예측 전에 모델 내부에서 동작시킨다.

그래프	노드	엣지	역할
Mention Graph (hMG)	entity mention (문장 내 등장 위치)	intra-entity / inter-entity / co-occurrence	Local 문맥 포착 Cross-sentence 연결
Entity Graph (EG)	entity canonical (통합 표현)	mention graph에서 압축된 엣지 + relation score	Global 관계 추론 Multi-hop path 탐색

그래프 생성 → GNN Message Passing 흐름

- 1 단계: entity_spans + token_hidden_states → mention node embedding 생성
- 2 단계: mention 간 edge 정의 (coreference / co-mention in same sentence / no relation)
- 3 단계: Mention Graph (hMG)에서 GAT message passing → mention representation 정제
- 4 단계: 동일 entity 의 mention 들을 attention 집계 → Entity Graph (EG) node 생성
- 5 단계: Entity Graph 에서 GCN/GAT 반복 → inter-sentence multi-hop 관계 포착
- 6 단계: refined entity_vectors → Graph Branch 의 relation score 계산

논문 근거: Zeng et al. (2020) GAIN Dev F1 57.28 (BERT-base) — GitHub: <https://github.com/PKUnlp-icler/GAIN>

4.2.3 Dual Branch Fusion

Direct Branch와 Graph Branch의 relation score를 결합하여 최종 예측에 활용한다. 각 branch의 강점을 살린다.

Branch	잘 잡는 관계	약한 관계
Direct Branch	Intra-sentence relation (문장 내 단서가 명확)	Multi-hop / cross-sentence relation
Graph Branch	Inter-sentence / multi-hop relation	단순하지만 locally mentioned relation
Fusion	가중합 또는 Gate mechanism으로 두 branch 결합	

- Fusion 방법 1 (Simple): $\text{score_final} = \alpha \cdot \text{score_direct} + (1 - \alpha) \cdot \text{score_graph}$ (α 학습 가능)
- Fusion 방법 2 (Gate): $\text{gate} = \sigma(W_g[\text{score_d}; \text{score_g}]) \rightarrow \text{score_final} = \text{gate} \odot \text{score_d} + (1 - \text{gate}) \odot \text{score_g}$
- Fusion 방법 3 (Concat): $[\text{score_direct}; \text{score_graph}] \rightarrow \text{Linear} \rightarrow \text{final logits}$ (구현이 가장 단순)

4.3 ATLOP + DREEM 강화

4.3.1 LongSumExp Pooling

기반 논문: ATLOP (Zhou et al., 2021) — Localized Context Pooling

Stage 1의 Mean Pooling을 LogSumExp Pooling으로 교체. mention 중 가장 강한 신호를 포착하며, smooth max approximation으로 작동한다.

4.3.2 Adaptive Threshold (ATLOP)

기반 논문: ATLOP (Zhou et al., 2021)

각 entity pair에 "threshold" class (TH)를 추가, relation score가 TH score보다 높은 relation만 채택. 클래스 불균형에 고정 threshold보다 강건한 성능.

4.3.3 DREEM Evidence Head

기반 논문: DREEM (Ma et al., 2023, EACL)

DREEM은 **relation 예측 시 근거 문장을 동시에 학습**하는 방법. evidence head를 추가하여 attention을 유도하고, evidence loss를 보조 supervision 신호로 활용한다.

DREEM 학습 파이프라인 (3 단계)

학습 단계	내용	데이터
1. Teacher 학습	Human-annotated data로 evidence 포함 모델 학습	train_annotated.json (gold evidence)
2. Silver Evidence 추론	Teacher가 distant data에 대해 token importance 추론	train_distant.json → .attns 파일
3. Student Self-training	Silver evidence로 student 학습 → human data fine-tuning	distant + annotated 순차 학습

Evidence Loss

```
L_total = L_RE + λ * L_evidence
L_RE     = Adaptive Threshold Loss (ATLOP)
L_evidence = KL-divergence(evidence_dist || predicted_attention)
λ         = 0.1 (BERT-base)
```

4.4 핵심 차이: Stage 2 vs Stage 3 & Stage 4

항목	Stage 2	Stage 3 & Stage 4
개선 대상	분류기 (Classifier) 강화	표현 (Representation) 자체 강화
핵심 모듈	ATLOP + DREEM	이기종 GNN (GAIN) / U-Net (DocuNet)
효과	Adaptive threshold + evidence supervision	문서 전체 entity 간 상호작용 + multi-hop 추론
코드 변경	relation_head 내부 수정	entity_repr → structural_encoder 삽입

5. 논문 기반 코드 구현 계획

레이어	기반 논문	참고 코드 / 구현 포인트
Preprocessing	DocRED (Yao 2019)	DocRED GitHub prepro.py 기반 커스터마이징
Encoder (V1)	Longformer (Beltagy 2020)	HuggingFace allenai/longformer-base-4096
Encoder (V2/V3)	RoBERTa (Liu 2019)	HuggingFace roberta-large + entity marker
SSAN-lite	SSAN (Xu 2021) arXiv:2102.10249	github.com/BenfengXu/SSAN — modeling_bert.py L267-280 Biaffine transform만 발췌, 상위 레이어만 적용
Entity Representation	ATLOP (Zhou 2021)	Logsumexp pooling, atlop GitHub
GAIN Double Graph	GAIN (Zeng 2020) arXiv:2009.13752	github.com/PKUnlp-icler/GAIN — MG/EG 생성 코드 GCN/GAT 기반 message passing
Fusion Head	Hybrid 설계	Linear concat 또는 Gating 방식 실험
Relation Extraction	DREEAM (Ma 2023)	Evidence-aware classifier, dreeam GitHub
Knowledge Graph	Neo4j Cypher	py2neo / neo4j-driver Python 클라이언트

6. 구현 계획 및 코드 구조

6.1 레이어별 기반 논문 및 참고 코드

레이어	기반 논문	참고 코드 / 구현 포인트
Preprocessing	DocRED (Yao 2019)	ATLOP GitHub prepro.py 기반
Encoder	BERT (Devlin 2019)	HuggingFace bert-base-uncased
Entity Repr.	ATLOP (Zhou 2021)	LogSumExp pooling (atlop GitHub)
Structural Reasoning	GAIN (Zeng 2020)	GCN/GAT message passing (gain GitHub)
Relation Head	ATLOP + DREEAM (Ma 2023)	Adaptive threshold + evidence head
Knowledge Graph	Neo4j	py2neo / neo4j-driver

6.2 프로젝트 디렉토리 구조

```

docre-kg/
├── data/
│   ├── docred/          # DocRED raw data
│   └── meta/            # rel2id.json, rel_info.json
├── src/
│   ├── preprocessing.py  # DocRED → feature 변환
│   ├── encoder.py       # BERT encoder wrapper
│   ├── entity_repr.py   # mention/entity pooling (mean → logsumexp)
│   ├── structural_encoder.py # 이기종 GNN / U-Net (Stage 3)
│   ├── relation_head.py # ATLOP + DREEM classifier
│   ├── model.py         # 전체 모델 forward (stage flag로 분기)
│   ├── postprocessing.py # threshold, dedup, normalization
│   └── kg_builder.py    # Neo4j 연동
├── scripts/
│   ├── train.py / evaluate.py / infer_evidence.py / build_kg.py
├── configs/
│   ├── stage1.yaml / stage2.yaml / stage3.yaml
└── README.md

```

6.3 model.py Forward 분기 설계

```

class DocREModel(nn.Module):
    def __init__(self, config):
        self.encoder = BertModel.from_pretrained('bert-base-uncased')
        self.entity_repr = EntityRepr(config)      # mean or logsumexp
        self.structural_encoder = None            # Stage 3에서만 활성화
        self.relation_head = RelationHead(config)  # fixed or ATLOP+DREEM

    def forward(self, batch, stage='stage1'):
        H = self.encoder(batch.input_ids, batch.attention_mask)
        entity_vecs = self.entity_repr(H, batch.entity_spans)

        if stage == 'stage3' and self.structural_encoder:
            # 이기종 GNN 또는 U-Net으로 entity vector 보정
            graph = build_hetero_graph(entity_vecs, batch.doc_info)
            entity_vecs = self.structural_encoder(entity_vecs, graph)

        logits = self.relation_head(entity_vecs, batch.entity_pairs)
        return logits # {relation_logits, evidence_logits (if stage2+)}

```

7. 핵심 참고 논문

1. Yao et al. (2019) DocRED: A Large-Scale Document-Level Relation Extraction Dataset. ACL 2019.
2. Wang et al. (2019) ATLOP: Document-Level Relation Extraction with Adaptive Thresholding and Localized Context Pooling. AAAI 2021.
3. Zhang et al. (2022) DREEM: Guiding Attention with Evidence for Document-Level RE. EMNLP 2022
→ GitHub: <https://github.com/YoumiMa/dreem>
4. Zeng et al. (2020) GAIN: Double Graph Based Reasoning for Document-level Relation Extraction. EMNLP 2020 → arXiv:2009.13752 → GitHub: <https://github.com/PKUnlp-icler/GAIN>
5. Xu et al. (2021) SSAN: Entity Structure Within and Throughout: Modeling Mention Dependencies for Document-Level RE. AAAI 2021 → arXiv:2102.10249 → GitHub: <https://github.com/BenfengXu/SSAN>
6. Beltagy et al. (2020) Longformer: The Long-Document Transformer. arXiv 2020.
7. Liu et al. (2019) RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv 2019.